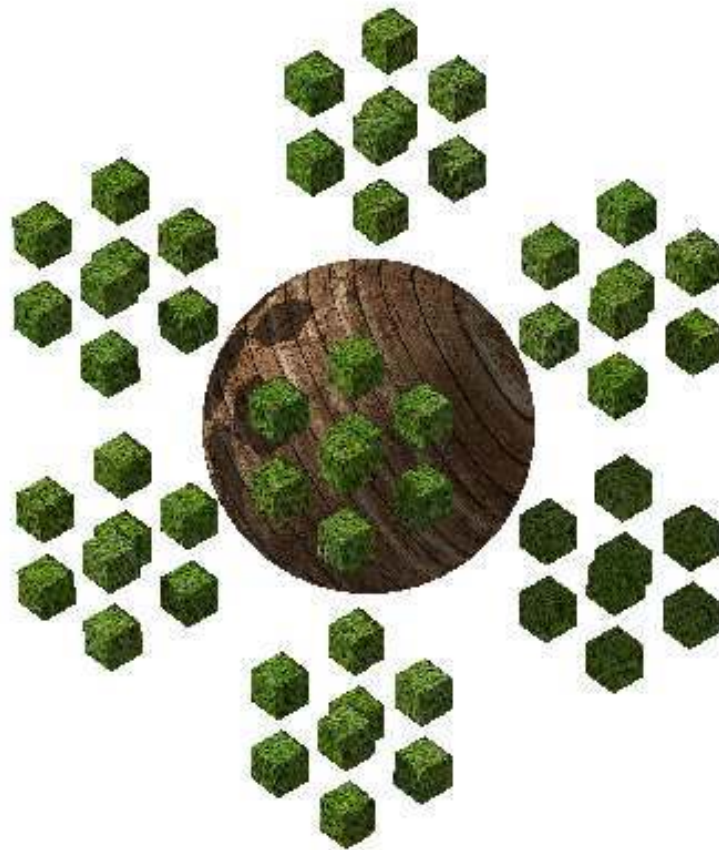




Computer Graphics - Inleiding

Tim Apers





Overzicht

- **Praktische informatie**
- **Vereisten voor de Grafische Engine**
- **Voorziene functionaliteit**
- **Tips**



Praktische Informatie

- **Tim Apers**
 - tim.apers@uantwerpen.be
- **Laurens De Wachter (student-assistant)**
 - Laurens.DeWachter@student.uantwerpen.be

- **Twee praktijksessies per week**
 - In principe: één voor elke groep
 - Iedereen is welkom tijdens beide sessies
 - Plaatsgebrek? → voorrang voor wie voor de sessie ingeschreven is
 - Tijdens de sessies
 - Uiteenzetting van de opdracht
 - Mogelijkheid om vragen te stellen
 - Aanwezigheid is *niet verplicht* (langskomen enkel voor uitleg of vraag is ok)
 - Neem **altijd** de cursus mee naar de practica
 - Lees de relevante hoofdstukken uit de cursus na **voor** het practicum!

- **Ontwikkelen van een grafische engine**
 - Voor het renderen van 3D afbeeldingen
- **De bestaande engine wordt telkens met nieuwe functionaliteit uitgebreid**
 - Oude functionaliteit moet blijven werken (**controleer dit regelmatig!**)
 - Uiteindelijke inzending: één engine die alle functionaliteit ondersteunt
 - Gegeven invoerbestand bepaalt welk type afbeelding gemaakt wordt

- **Praktijk (10-12/20)**

- 2D L-Systemen (1,25)
- 3D Lijntekeningen (1)
- 3D Lichamen (1)
- Z-Buffering met lijnen (1)
- Z-Buffering met driehoeken (1,5)
- 3D Fractalen (1)
- Clipping (0,5)
- Belichting (1,5)
- Schaduw (1)
- Texture mapping (1,5)
- 3D Lijntekeningen met bollen & cilinders (0,75)

Opdrachten tot hier (goed) geïmplementeerd
volstaat voor slaagcijfer

- **Ongeveer één opgave per week**
 - Elke opgave bestaat uit
 - Eén of meerdere deelopdrachten
 - Meestal één grote en enkele kleine opdrachten
 - Te verdienen punten aangegeven per deelopdracht
 - » Aantal punten geeft aan hoe belangrijk de deelopdracht is, niet noodzakelijk hoe uitdagend het is
 - Eventuele extra oefeningen
 - Niet bij elke opdracht
 - Enkel voor jezelf
 - Hoeven *niet* te worden ingediend
 - Worden *niet* gequoteerd

- **Proefevaluatie van de grafische engine**
 - NIEUW
 - Proefevaluaties verlopen via INGINious
 - 1 indiening per uur toegestaan
 - Staan niet op punten, wel deadlines!
 - Pixel matcher, hoger dan 99% is goed
 - ⇒ Stuur gerust een mail als je denkt dat INGINious fout is!
 - Deelname aan de proefevaluaties is *niet* verplicht ...
 - ... maar wel **ten sterkste aangeraden!**
 - Verdere vragen en/of problemen
 - Praktijksessies
 - E-mail



Planning

Week	Datum	Opgave	Deadline Proefevaluatie
1	14/02 – 18/02	Inleiding	–
2	21/02 – 25/02	2D L-Systemen	–
3	28/02 – 04/03	–	–
4	07/03 – 11/03	3D Lijntekeningen	2D L-Systemen
5	14/03 – 18/03	3D Lichamen	–
6	21/03 – 25/03	Z-Buffering met lijnen	3D Lijntekeningen
7	28/03 – 01/04	Z-Buffering met driehoeken	3D Lichamen
-	04/04 – 15/04	– (Paasvakantie)	Z-buffering met lijnen
8	18/04 – 22/04	3D Fractalen	Z-Buffering met driehoeken
9	25/04 – 29/04	Belichting	3D Fractalen
10	02/05 – 06/05	Schaduw & Texture Mapping	Belichting
11	09/05 – 13/05	Bollen & Cilinders	Schaduw & Texture Mapping
12	16/05 – 20/05	–	Bollen & Cilinders
13	23/05 – 27/05	–	–

Deadlines proefevaluatie zijn altijd de zondagavond om 23u59.
Er is dus altijd minstens een volledige week (incl. twee weekends)
tussen het practicum en de bijbehorende deadline

- **Plan zélf wanneer je wat doet**
 - Hou rekening met deadlines van andere vakken
 - **Onderschat het werk niet!**



Vereisten voor de Grafische Engine

- **Programmeertaal: C++**
 - Ontwikkelomgeving is volledig vrij te kiezen:
 - CLion, Makefiles, Eclipse, Visual Studio, ...
 - Enkele bestanden met essentiële functionaliteit al beschikbaar op BlackBoard
 - Op het eind van het semester moet je engine onder **Linux** met **CMake** werken!
 - In principe: Ubuntu 18.04 LTS 64-bit, gcc 8
 - De Labo PC's
 - INGINious
 - We gebruiken standaard C++14, de default van gcc 8. Wie gebruik wil maken van een andere versie (omwille van compatibiliteitsproblemen of om recente features te gebruiken) laat ons eerst iets weten.



Algemene vereisten

- Voor de finale evaluatie worden er met elke engine tussen de 200 en 300 images gegenereerd
- Je grafische engine mag:
 - maximaal **2 GB** geheugen gebruiken
 - Maximaal geheugengebruik met onze engine < 750 MB
 - maximaal **30 min** rekenen per image (op de labo pc's)
 - Maximale rekentijd met onze engine < 1,5 mins



Algemene vereisten

- Opgaves moeten *individueel* worden gemaakt
 - Overleggen mag
 - Plagiaat = Fraudecommissie
 - Code kopiëren (van elkaar en online) is *ten strengste verboden*
 - Zonder toestemming van ons mogen er **geen** libraries worden gebruikt
 - Alles wat je in principe nodig hebt wordt door ons voorzien

INI bestand

- Beschrijft het type van de te renderen afbeelding, en de precieze inhoud ervan
- Door ons voorzien

BMP afbeelding

- Bevat de gerenderde afbeelding
- Exact 1 per INI bestand
- Heeft dezelfde naam als het invoerbestand (op extensie na)



Engine



Input en Output formaat

- **Input: INI-configuratiebestand**
 - Beschikbaar op BlackBoard
 - Eén ini-bestand per te genereren image
 - Bijbehorend telkens een png-bestand met verwachte uitkomst
 - Je resultaat hoeft niet pixel-perfect overeen te komen met het voorbeeld (contacteer ons bij twijfel!)
 - Bestandsformaat door ons gespecificeerd
 - Indien niet vermeld in de opgave: formaat vrij te kiezen
 - Voorzie zélf de nodige configuratiebestanden
 - Geen configuratiebestand == geen punten
- **Output: wegschrijven naar BMP-bestand**



Aanspreken van de Engine

- **Interface: engine.cc** (beschikbaar op BlackBoard)
 - Implementeert:

```
int main(int argc, char* argv[]);
```
 - Zorgt voor:
 - Het inlezen van INI-bestanden
 - Het wegschrijven van BMP-bestanden
 - Laat ons toe om **elke** engine op exact dezelfde manier te gebruiken
 - Verplicht te gebruiken!
 - Pas enkel **generate_img** aan



Engine builden via command line

Tip: maak een “buildmap” voor alle gegenereerde files zodat die makkelijk op te kuisen zijn (bv. wanneer je gaat insturen).

Engine builden en runnen (Linux):

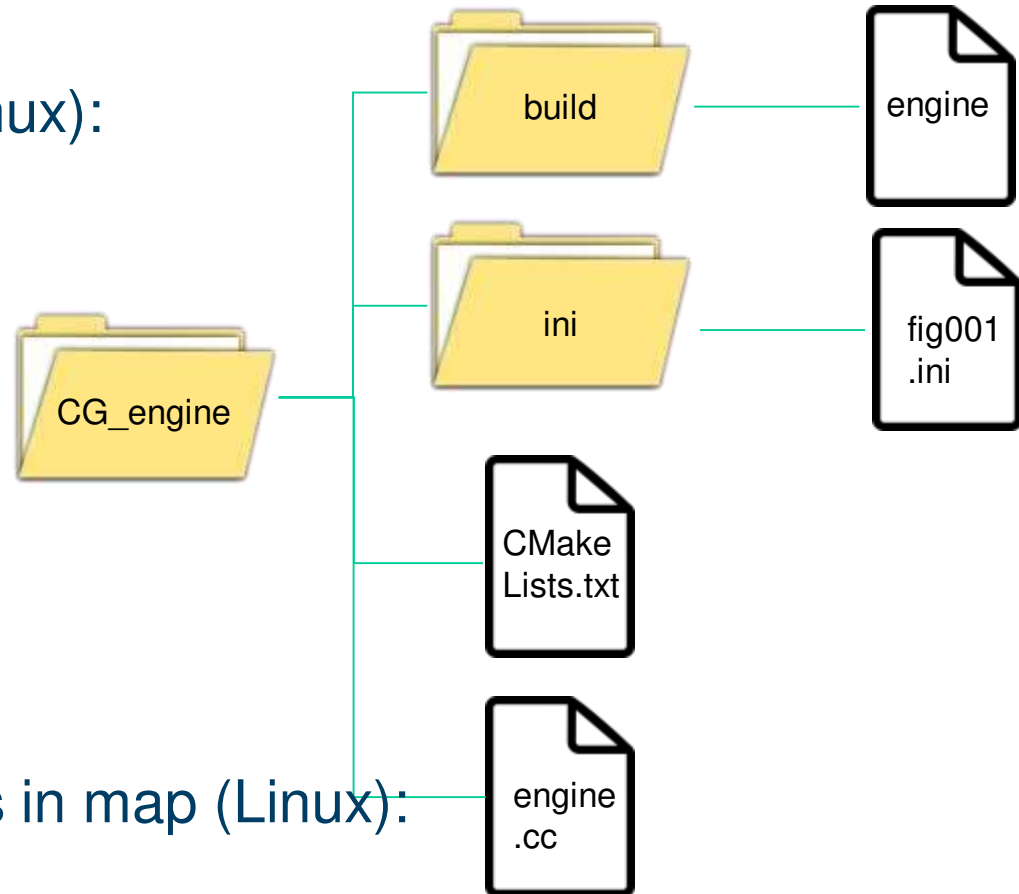
```
CG_engine$ mkdir build
CG_engine$ cd build
build$ cmake ..
build$ make
build$ ./engine
```

Buildfiles verwijderen (Linux):

```
CG_engine$ rm -rf build
```

Engine draaien op alle ini files in map (Linux):

```
ini$ ../build/engine *.ini
```

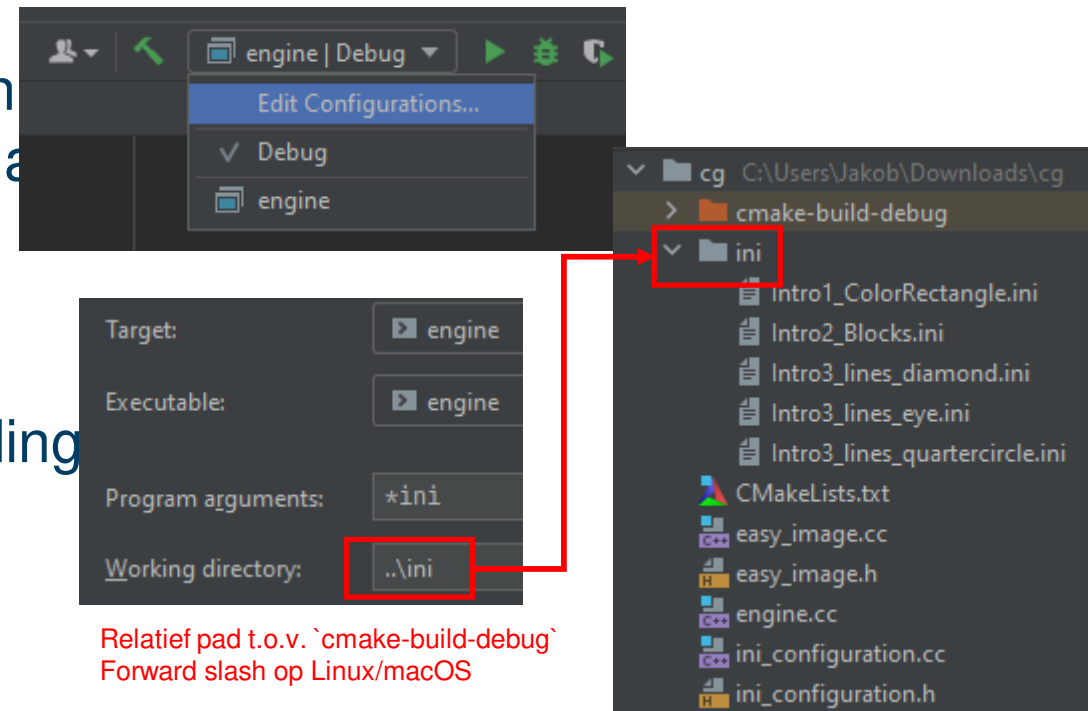


Engine runnen via CLion

Open de map met je project in CLion
Voorzie een submap met daarin je ini-files

Stel rechtsboven een Configuration
Kies als working directory de submap
met je ini's,
en als argument : *ini

Zo genereert je engine alle afbeelding
die in de map beschreven worden



Argument *ini werkt op sommige systemen niet
Laat dan het argument leeg en voorzie in de map het bestand `filelist`
dat lijn per lijn elke ini-file opsomt.
Deze `filelist` is op BlackBoard telkens voorzien



Vereisten voor de inzendingen

- **Eén bestand** (`.zip`) met daarin:
 - Alle source-files die nodig zijn om de engine te compileren
 - Een 'README'-bestand overzicht van wat geïmplementeerd is
 - Voorbeeld op BlackBoard
 - **Geen** `CMakeLists.txt` om de engine mee te compileren
 - **Geen** ini-files behalve voor:
 - Opdrachten waar dit specifiek wordt gevraagd
 - Functionaliteit waarvoor geen inputformaat beschreven is
 - Voorzie in het 'README'-bestand extra uitleg
 - **Geen** gegenereerde images
 - **Geen** gecompileerde code
- **Geen** `.rar`,... file (`.tgz` wel ok)

- **Voor elke opgave:**
 - Staat er een PDF-bestand op blackboard
 - Beschrijft *wat* je moet doen
 - Beschrijft het ini-bestandsformaat in detail
 - Aan dit document **moet** je je houden bij het implementeren van je engine!
 - Wordt er een presentatie gegeven
 - Deze dient enkel om de opdracht te verduidelijken
 - Deze bevat meestal C++ code fragmenten
 - Deze dienen ter illustratie
 - Deze voldoen niet altijd aan de regels van de kunst
 - Je mag je hierop baseren als je dat nuttig vindt
 - Je bent **NIET** verplicht om de gegeven code-fragmenten te gebruiken



Voorziene functionaliteit



Voorziene functionaliteit

- `ini::Configuration`
 - Inlezen van INI-configuratiebestanden
- `img::EasyImage`
 - Eenvoudige Image Library
- **engine.cc**
 - ‘Standaard’ interface voor het aanspreken van de grafische Engine
 - Verplicht te gebruiken
- `LParser::LSystem2D`
 - Klasse om L-Systemen te parsen
(wordt uitgelegd bij de opdracht 2D-Lsystemen)
- `Matrix & Vector3D`
 - Eenvoudige maar krachtige Matrix Library
(wordt uitgelegd bij de opdracht 3D Lijntekeningen)

- **Voorbeeld configuratiebestand**

```
[DitIsEenSectie]
key = 1234
andereKey = 'Hello world!' ; Dit is commentaar
reeleWaarde = 3.14 ; pi
```

```
[DitIsEenAndereSectie]
booleanWaarde = TRUE
stringValue = 'Dit is de waarde...'
ookEenString = bla bla
reeleTupleWaarde = (1.0, 5.0, 0.4)
```



`ini::Configuration`

- **Elk ini-bestand bevat *altijd* een sectie** `General`
 - Deze sectie bevat *altijd* een waarde `type`

```
[General]  
type="<type>"
```
 - Dient om onderscheid te maken tussen de verschillende opdrachten
- **INI-bestanden worden ingelezen door de klasse** `ini::Configuration`
- **Drie soorten methodes om parameters op te vragen:**
 - `.as_<type>_or_die()` ;
 - `.as_<type>_if_exists()` ;
 - `.as_<type>_or_default()` ;



```
ini::Configuration[] []  
    .as_<type>_or_die();
```

- Voor waarden die in een geldige configuratie *moeten* voorkomen.
- Als de waarde niet bestaat of een verkeerd type heeft, stopt het programma.

```
int main()  
{  
    std::ifstream fin("file.ini");  
    ini::Configuration conf(fin);  
  
    std::string type=  
        conf["General"]["Type"].as_string_or_die();  
    int intVal =  
        conf["Section1"]["intVal"].as_int_or_die();  
    std::vector<double> doubleTuple =  
  
    conf["Section1"]["tuple"].as_double_tuple_or_die();  
  
    return 0;  
}
```

```
[General]  
Type="Hello World"  
[Section1]  
intVal = 1  
tuple= (1, 3.14, 5)
```



```
ini::Configuration[] []  
    .as_<type>_or_die();
```

- Voor waarden die in een geldige configuratie *moeten* voorkomen.
- Als de waarde niet bestaat of een verkeerd type heeft, stopt het programma.
- `ini::Configuration` kan zélf naar het juiste type casten

```
int main()  
{  
    std::ifstream fin("file.ini");  
    ini::Configuration conf(fin);  
  
    std::string type = conf["General"]["Type"];  
    int intVal = conf["Section1"]["intVal"];  
    std::vector<double> doubleTuple =  
conf["Section1"]["tuple"];  
    return 0;  
}
```

```
[General]  
Type="Hello World"  
[Section1]  
intVal = 1  
tuple= (1, 3.14, 5)
```



```
ini::Configuration[] []  
    .as_<type>_if_exists ();
```

- Voor waarden die niet per se aanwezig moeten zijn.
- Waarde wordt opgeslagen in de opgegeven parameter
- **bool** return waarde: geeft aan of de waarde bestaat

```
int main()  
{  
    std::ifstream fin("file.ini");  
    ini::Configuration conf(fin);  
  
    int intVal;  
    if(conf["Section1"]["intVal"].as_int_if_exists(intVal))  
    {  
        //do something useful here  
    }  
    return 0;  
}
```

```
[General]  
Type="Hello World"  
[Section1]  
intVal = 1  
tuple= (1, 3.14, 5)
```



```
ini::Configuration[] []  
    .as_<type>_or_default();
```

- Voor waarden die niet per se aanwezig moeten zijn.
- Als de waarde niet bestaat, wordt een *default* waarde teruggegeven.

```
int main()  
{  
    std::ifstream fin("file.ini");  
    ini::Configuration conf(fin);  
  
    int intVal =  
        conf["Section4"]["blabla"].as_int_or_default(0);  
    return 0;  
}
```

```
[General]  
type="Hello World"  
[Section1]  
intVal = 1  
tuple= (1, 3.14, 5)
```



img::EasyImage

- **Beschikbare methodes:**

```
class EasyImage {  
    public:  
        EasyImage (unsigned int width, unsigned int height,  
                    Color bgColor = Color());  
  
        unsigned int get_width() const;  
        unsigned int get_height() const;  
  
        // Function operator, This operator returns a reference  
        // to a particular pixel of the image.  
        Color& operator() (unsigned int x, unsigned int y);  
  
        void clear(Color bgColor = Color());  
  
        // Draws a line from pixel (x0,y0) to pixel  
        // (x1,y1) in the specified color  
        void draw_line(unsigned int x0, unsigned int y0,  
                        unsigned int x1, unsigned int y1,  
                        Color color);  
};
```

- Voorbeeld

```
#include <iostream>
#include <fstream>
#include "easy_image.h"

int main()
{
    img::EasyImage image(256,256);
    for(unsigned int i = 0; i < 256; i++)
        for(unsigned int j = 0; j < 256; j++)
        {
            image(i,j).red = i;
            image(i,j).green = j;
            image(i,j).blue = (i+j)%256;
        }
    std::ofstream fout("out.bmp", std::ios::binary);
    fout << image;
    fout.close();
    return 0;
}
```



- **Interface tot de grafische engine**
 - Wordt door ons gebruikt om meerdere images te genereren
 - Verplicht te gebruiken
- **Implementeer zélf de volgende functie:**
 - `img::EasyImage generate_image(const ini::Configuration&);`



Tips



Enkele nuttige tips

- **Memory management**
- **Gegevensstructuren**
- **String manipulaties**
- **Afronden van getallen**
- **C++ Development**
- **C++ Coding Style Guide**
- **Algemene misverstanden**



Geheugen management

- **C++ heeft geen garbage collector (zoals Python)**
 - Alles wat je zélf **new**t moet je zélf **delete**n
 - Pas op voor memory leaks
 - Beperkte hoeveelheid geheugen
 - Gebruik `valgrind` om memory leaks te vinden
 - Gebruik smart pointers
- **Pas op met het doorgeven van argumenten:**
 - `World3D`: +- 500 MB groot
 - `img::EasyImage render_image(const World3D fig);`
 - De datastructuur wordt gekopieerd bij het oproepen van de functie
 - Bij elke oproep: 500MB extra geheugen nodig
 - `img::EasyImage render_image(const World3D &fig);`
 - De functie gebruikt de originele datastructuur



Beschikbare Gegevensstructuren

- In C++ worden er een heleboel datastructuren voorzien
 - het is NIET nodig om deze zélf te implementeren
- `std::vector`

```
#include <vector>
```

```
int main()  
{  
    //een vector vervangt gewone arrays  
    std::vector<double> dbl_vector(10, 0.0);  
    for(int i = 0; i < dbl_vector.size(); i++)  
    {  
        //[]-operator doet geen boundary-checks  
        dbl_vector[i] = i;  
        //.at(...) doet wél boundary checks  
        dbl_vector.at(i) = i;  
    }  
    //een vector kan dynamisch groeien/krimpen  
    dbl_vector.push_back(11.0);  
  
    return 0;  
}
```



Beschikbare Gegevensstructuren

- In C++ worden er een heleboel datastructuren voorzien
 - het is NIET nodig om deze zélf te implementeren

- `std::list`

```
#include <list>
```

```
int main()
```

```
{  
    //std::list voorziet een gelinkte lijst  
    std::list<double> dbl_list();  
    dbl_list.push_back(1.0);  
    dbl_list.push_back(2.0);  
  
    for(std::list<double>::iterator i = dbl_list.begin();  
        i != dbl_list.end();  
        i++)  
    {  
        double &d = *i;  
    }  
  
    return 0;  
}
```



Beschikbare Gegevensstructuren

- **In C++ worden er een heleboel datastructuren voorzien**
 - het is NIET nodig om deze zélf te implementeren
- **Hou rekening met structuurgerelateerde performantie**
 - Vector: insert/delete in het midden of vooraan = traag
 - List: n-de element opvragen = traag
 - <https://github.com/gibsjose/cpp-cheat-sheet/blob/master/Data%20Structures%20and%20Algorithms.md>
- **Andere interessante containers**
 - `std::map`
 - `std::set`
- **Documentatie:**
 - <http://en.cppreference.com/w/>



Uitvoeren van String Manipulaties (C++11)

- **std::string**

```
#include <string>
#include <iostream>

int main()
{
    //Omzetten van strings naar andere types
    int i = std::stoi("-24");
    std::cout << i << std::endl;

    unsigned int ui = std::stoul("123");
    std::cout << ui << std::endl;

    double d = std::stod("3.14");
    std::cout << d << std::endl;

    //Types omzetten naar strings
    std::string str = std::to_string(2.78);
    std::cout << str << std::endl;

    //Strings opstellen
    std::string str1 = "Figure";
    std::string str2 = std::to_string(3);
    std::string combine = str1 + str2;
    std::cout << combine << std::endl;

    return 0;
}
```

Output:

```
-24
123
3.14
2.780000
Figure3
```



Afronden van getallen

```
#include <iostream>
#include <cmath>

int main()
{
    for (double d = 1.0; d < 2.1; d += 0.1)
    {
        int i = (int) d;
        std::cout << "d: " << d
                    << "\ti: " << i << std::endl;
    }
}

int main()
{
    for (double d = 1.0; d < 2.1; d += 0.1)
    {
        int i = (int) d;
        std::cout << "d: " << d
                    << "\ti: " << i << std::endl;
    }
    std::cout << std::endl;
    return 0;
}
```

Gebruik **altijd** lower cast om
doubles naar **ints** te converteren!

d: 1		i: 1
d: 1.1	i: 1	
d: 1.2	i: 1	
d: 1.3	i: 1	
d: 1.4	i: 1	
d: 1.5	i: 1	
d: 1.6	i: 1	
d: 1.7	i: 1	
d: 1.8	i: 1	
d: 1.9	i: 1	
d: 2		i: 2

d: 1		i: 1
d: 1.1	i: 1	
d: 1.2	i: 1	
d: 1.3	i: 1	
d: 1.4	i: 1	
d: 1.5	i: 2	
d: 1.6	i: 2	
d: 1.7	i: 2	
d: 1.8	i: 2	
d: 1.9	i: 2	
d: 2		i: 2

Afrondingsfouten

- Vermijd typische afrondingsfouten:
 - Gebruik enkel een **int** als de variabele **nooit** een reële waarde *mag* bevatten
 - Gebruik **ints** voor: array-indices, pixel-coördinaten, ...
 - Gebruik **doubles** voor: afstanden, matrix bewerkingen, hoeken, ...
 - Rond een getal pas af wanneer het echt nodig is
 - Dus niet: **int** `x = radius * std::lround(std::sin(a));`
 - Maar wel: **int** `x = std::lround(radius * std::sin(a));`
 - Gebruik **doubles** en geen **floats**



C++ Development tips

- **C++ is niet de meest eenvoudige taal**
 - Gebruik de tools die beschikbaar zijn
 - Debugger: `gdb` / CLion debugger / Visual Studio / ...
 - Memory management: `valgrind`
 - Gebruik onderstaande compile-flags (clang, g++)
 - `-Wall -Wextra`
 - » zet alle compiler-warnings aan
 - `-fstack-protector-all`
 - » beschermt tegen bepaalde 'out-of-bounds' fouten
 - `-g3`
 - » compileer met maximale debug-informatie

(Omdat de flags niet bij elke compiler ondersteund zijn, worden ze niet standard gebruikt in onze CMakeLists.txt. Wel staat aangegeven waar je ze kunt toevoegen.)



C++ Coding Style Guide

- **Werk objectgeoriënteerd**
 - Probeer zoveel mogelijk functionaliteit in correcte classes te implementeren
- **Elke class heeft zijn eigen .h en .cc file**
 - Elke class heeft 2 files: een .h en een .cc (of .cpp) file
 - Geef deze files dezelfde naam als de class
- **Lees meer:**
 - <https://google.github.io/styleguide/cppguide.html>

Bij de evaluatie wordt geen rekening gehouden met codekwaliteit.
Bij moeilijk leesbare code is het wel mogelijk dat we geen hulp kunnen bieden bij problemen.



Algemene misverstanden

- **Zorg voor een correcte omzetting tussen graden en radialen**
 - `std::sin` en `std::cos` maken gebruik van **radialen**
 - Hoeken worden in de opgaves in **graden** opgegeven
- **Zorg dat kleuren correct worden geschaald**
 - `img::Color`: gehele RGB-kleurcomponenten tussen 0 en 255 (incl.)
 - In de INI-bestanden: reële kleurcomponenten tussen 0 en 1 (incl.)
- **Elke opgave moet individueel kunnen worden getest**
 - Vb: Na het implementeren van belichting moet zuivere Z-buffering (zonder belichting) nog werken
 - Tip: test elke keer een nieuwe opgave af is of alle voorgaande nog correct werken. Als oude functionaliteit bij je finale implementatie niet meer werkt, kunnen we er ook niet op rekenen dat het werkt

Hulp nodig?

- Stuur een mailtje naar beide assistenten
- Beschrijf bondig maar duidelijk wat er fout gaat
- Beschrijf bondig hoe je het probleem al hebt proberen op te lossen
- Bij codegerelateerde problemen: stuur je *volledige* engine in bijlage
 - In een .zip net zoals inzendingen voor proefevaluatie
 - Vermeld waar relevant regelnummers van code
- Voorzie één of meerdere ini-files waarbij het probleem zich voordoet (zelfs al komen ze van BlackBoard)

